Итог

Статус: Не готово к сдаче.

Реализация по коду охватывает почти всю обязательную часть T3: CIFAR-10, Vanilla GAN, DCGAN, единый цикл обучения, loss-графики, визуальные сэмплы, latent space interpolation, FID, Inception Score, checkpoints и итоговое сравнение. Архитектуры и отдельный training batch прошли статические и локальные smoke-проверки. Однако notebook сохранён без результатов выполнения: у всех 23 кодовых ячеек execution_count равен None, а outputs отсутствуют. Кроме того, по умолчанию установлен FAST_MODE=True, поэтому обычный Run all выполняет только 1 эпоху на 2 048 изображениях и считает отладочные метрики на 1 000 изображениях, тогда как финальный режим в самом ноутбуке требует 100 эпох, полного training split и 10 000 изображений для FID/IS. До сдачи нужно запустить FULL_MODE, сохранить все результаты и проверить фактическое качество генерации.

Матрица соответствия

№	Источник	Требование	Статус	Доказательство	Замечание
1	T3	Изучить принципы GAN и сравнить Vanilla GAN с DCGAN.	частично	Markdown-ячейки 2-4 и кодовые ячейки 17-49 реализуют обе архитектуры и сравнение.	Код есть, но фактическое сравнение не подтверждено outputs.
2	T3 / инструменты	Использовать Python 3.9+ и PyTorch либо TensorFlow.	выполнено	Ячейки 7-8: устанавливаются и импортируются torch и torchvision; вся реализация написана на PyTorch.	Требуемый стек соблюдён.
3	T3	Загрузить CIFAR-10 или допустимый альтернативный датасет.	выполнено	Ячейки 10-11: datasets.CIFAR10 загружает официальные train и test splits.	Выбран основной рекомендованный датасет CIFAR-10.
4	T3	Нормализовать изображения в диапазон [-1, 1].	выполнено	cifar10_transform использует ToTensor и Normalize((0.5,...),(0.5,...)); диапазон проверяется assert в ячейке 11.	Предобработка согласована с Tanh генераторов.
5	T3	Визуализировать 9-16 случайных исходных изображений.	частично	Ячейка 14 формирует сетку из 16 изображений и подписывает классы.	Код есть, но визуальный output не сохранён.
6	T3	Проверить распределение классов CIFAR-10.	частично	Ячейка 13 строит таблицу и barplot по полному training split.	Фактическая таблица и график в notebook отсутствуют.
7	T3	Реализовать Vanilla GAN на полносвязных слоях.	выполнено	Ячейки 16-18: VanillaGenerator и VanillaDiscriminator с Linear, ReLU, LeakyReLU, BatchNorm и Tanh.	Локальная проверка форм тензоров прошла.
8	T3	Реализовать DCGAN на свёрточных слоях.	выполнено	Ячейки 26-28: ConvTranspose2d генератор и Conv2d дискриминатор.	Локальная проверка форм тензоров прошла.
9	T3 / методика	Использовать BatchNorm, ReLU/LeakyReLU и Tanh в DCGAN.	выполнено	Ячейка 27 содержит BatchNorm2d, ReLU, LeakyReLU(0.2) и финальный Tanh.	Архитектура соответствует рекомендациям.
10	T3 / методика	Использовать BCEWithLogitsLoss без отдельного Sigmoid.	выполнено	Ячейки 4, 18 и 28: nn.BCEWithLogitsLoss; финальные слои дискриминаторов возвращают logits.	Численно устойчивый вариант реализован корректно.
11	T3 / методика	Использовать Adam с lr=2e-4, beta1=0.5 и beta2=0.999.	выполнено	Ячейки 8, 18, 22 и 31 задают LEARNING_RATE=2e-4 и ADAM_BETAS=(0.5,0.999).	Настройки одинаковы для обеих моделей.
12	T3 / методика	Использовать batch size 64-128 и 100-200 эпох для финального CIFAR-10 эксперимента.	частично	BATCH_SIZE=64 и FULL_EPOCHS=100 заданы в ячейке 8.	По умолчанию FAST_MODE=True, поэтому выполняется только 1 эпоха на подмножестве.
13	T3	Обучить Vanilla GAN.	частично	Ячейки 21-22 содержат полный цикл и вызов train_gan.	Нет журнала фактического обучения; текущий режим по умолчанию отладочный.
14	T3	Обучить DCGAN в сопоставимых условиях.	частично	Ячейки 30-31 используют тот же train_gan, seed, DataLoader, epochs, batch size и Adam.	Нет сохранённого результата выполнения.

№	Источник	Требование	Статус	Доказательство	Замечание
15	ТЗ	Отследить loss генератора и дискриминатора.	частично	train_gan сохраняет loss_G и loss_D каждого шага; ячейки 24, 25, 33 и 35 строят графики.	Графики и численные значения отсутствуют в outputs.
16	ТЗ	Визуализировать сгенерированные изображения в ходе обучения.	частично	train_gan создаёт snapshots на эпохах 0, середине и конце; plot_sample_snapshots и plot_final_sample_comparison строят сетки.	Сетки не сохранены в приложенном notebook или отдельном файле.
17	ТЗ	Рассчитать FID на достаточном числе real и generated images.	частично	Ячейки 41-43 используют torch-fidelity; FULL_MODE задаёт 10 000 real и 10 000 generated images на модель.	По умолчанию используется 1 000 изображений и нет фактического значения FID.
18	ТЗ	Рассчитать Inception Score.	частично	calculate_fid_and_is включает isc=True и сохраняет mean и std.	Фактические IS и IS std отсутствуют.
19	ТЗ / качество	Сравнивать модели единым pipeline и одинаковым числом samples.	выполнено	Ячейки 41-46 используют один real set, одинаковое число generated images, seed и функцию метрик.	Методика сравнения по коду единообразна.
20	ТЗ	Выполнить latent space interpolation.	частично	Ячейки 37-40 строят общий путь из 10 точек для Vanilla GAN и DCGAN.	Проверка крайних точек реализована, но изображения интерполяции не сохранены.
21	ТЗ / рекомендация	Сохранять checkpoints моделей.	частично	train_gan сохраняет состояния G, D и оптимизаторов для обеих моделей в lab4_gan_runtime/checkpoints.	Фактические файлы не приложены; сохраняется последняя эпоха, а не лучшие веса по визуальному качеству.
22	ТЗ	Сравнить качество, устойчивость и вычислительную стоимость моделей.	частично	Ячейки 34-36 и 45-49 сравнивают loss, samples, FID, IS, время и число параметров.	Таблица и итоговый вывод формируются только во время запуска и сейчас отсутствуют.
23	ТЗ	Дать визуальную оценку качества и проанализировать mode collapse.	не выполнено	Markdown-ячейки 36, 40 и 47 объясняют критерии анализа.	Фактические сэмплы отсутствуют, поэтому конкретный визуальный анализ и вывод о mode collapse не сделаны.
24	ТЗ / опционально	Реализовать WGAN-GP, Conditional GAN или StyleGAN2-ADA.	не выполнено	В notebook реализованы только Vanilla GAN и DCGAN.	Пункт опциональный и на обязательную приёмку не влияет.
25	ТЗ / опционально	Сравнить с DDPM или VAE.	не выполнено	Код DDPM и VAE отсутствует.	Пункт опциональный и на обязательную приёмку не влияет.
26	ТЗ / отчёт	Описать введение, датасет, методологию, результаты и выводы.	частично	Notebook содержит 19 последовательных разделов и пояснения Markdown.	Структура полная, но раздел результатов не содержит выполненных outputs.
27	ТЗ / формат	Сдать Jupyter Notebook с Markdown либо PDF и ссылку на воспроизводимый Colab.	выполнено	practical_work_4_gan.ipynb имеет формат nbformat 4.5, 50 ячеек, включая 27 Markdown и 23 code.	Формат notebook соблюдён; для защиты всё равно нужны сохранённые результаты.
28	ТЗ / качество	Зафиксировать случайность.	выполнено	set_reproducible_seed фиксирует random, NumPy, PyTorch, CUDA и cuDNN; DataLoader получает отдельный generator с seed.	Базовая воспроизводимость реализована.
29	ТЗ / среда	Поддерживать запуск в Google Colab с GPU или локальном Jupyter с CUDA.	выполнено	DEVICE автоматически выбирает CUDA; pin_memory и num_workers зависят от доступности GPU.	Код не привязан к конкретной локальной машине.
30	ТЗ / рекомендация	Регулярно сохранять checkpoints и samples на Google Drive.	не выполнено	drive.mount и пути Google Drive отсутствуют; samples только отображаются, checkpoints сохраняются в runtime-каталог.	Это сформулировано в ТЗ как рекомендация, а не обязательное условие.
31	Качество	Не хранить секреты и внешние доступы.	выполнено	Статический поиск не выявил API keys, tokens, passwords, drive.mount, W&B или Roboflow.	Секреты отсутствуют.
32	Качество	Предоставить доказательства фактического выполнения.	не выполнено	Все 23 кодовые ячейки имеют execution_count=None; суммарное число outputs равно 0.	Главный блокер приёмки.
33	Качество	Run all должен формировать финальный, а не отладочный результат.	не выполнено	В ячейке 8 задано FAST_MODE=True, FAST_EPOCHS=1, FAST_SUBSET_SIZE=2048 и FAST_EVAL_IMAGES=1000.	Без ручного изменения флага стандартный запуск не выполняет финальный эксперимент.
34	Качество	Базовые функции и формы тензоров должны быть исполнимы.	выполнено	AST всех 23 code-ячеек прошёл; локально проверены формы Vanilla/DCGAN, один training batch и latent interpolation.	Проверки прошли без загрузки датасета и полного обучения.
35	Спецификация / тикеты	Проверить дополнительные документы проекта.	не проверено	По прямому указанию пользователя аудит ограничен двумя приложенными notebook-файлами.	Другие файлы и папки не открывались; это не считается ошибкой.

Найденные проблемы

AUD-001 - Critical

Источник: Качество / доказательства выполнения

Описание: Notebook сохранён без единого результата выполнения. Нельзя подтвердить обучение Vanilla GAN и DCGAN, loss-графики, качество generated samples, latent interpolation, FID, Inception Score, checkpoints и итоговую сравнительную таблицу.

Как воспроизвести или проверить: Открыть practical_work_4_gan.ipynb и проверить execution_count и outputs всех code-ячеек.

Ожидаемый и фактический результат: Ожидается выполненный notebook с логами, таблицами, графиками и изображениями. Фактически у 23 из 23 code-ячеек execution_count=None, outputs отсутствуют.

Рекомендуемое исправление: Выполнить notebook целиком в Colab или Jupyter с GPU и сохранить его вместе со всеми outputs.

AUD-002 - Critical

Источник: ТЗ / финальный эксперимент

Описание: По умолчанию включён FAST_MODE=True. Обычный Run all обучает модели только 1 эпоху на 2 048 изображениях и рассчитывает FID/IS по 1 000 изображениям, хотя ТЗ указывает 100-200 эпох и не менее 10 000 изображений для FID.

Как воспроизвести или проверить: Проверить константы FAST_MODE, FAST_EPOCHS, FULL_EPOCHS, FAST_SUBSET_SIZE, FAST_EVAL_IMAGES и FULL_EVAL_IMAGES в ячейке 8.

Ожидаемый и фактический результат: Ожидается финальный запуск на полном training split, 100 эпохах и 10 000 real/generated images. Фактически сохранённая конфигурация по умолчанию является только debug-режимом.

Рекомендуемое исправление: Перед финальным запуском установить FAST_MODE=False, выполнить Run all и сохранить полученные outputs.

AUD-003 - Major

Источник: ТЗ / анализ результатов

Описание: В notebook описаны правила визуального анализа, но отсутствуют фактические изображения и основанные на них выводы о чёткости, разнообразии, плавности latent interpolation и возможном mode collapse.

Как воспроизвести или проверить: Выполнить ячейки 25, 33, 35 и 39 и проверить последующие текстовые выводы.

Ожидаемый и фактический результат: Ожидается анализ реально полученных сеток. Фактически присутствуют только общие инструкции о том, как их интерпретировать.

Рекомендуемое исправление: После полного запуска добавить в вывод конкретные наблюдения, относящиеся к отображённым Vanilla GAN и DCGAN samples.

AUD-004 - Major

Источник: ТЗ / метрики качества

Описание: Фактические FID и Inception Score отсутствуют, а debug-настройка использует только 1 000 изображений, что ниже указанного в ТЗ минимального объёма для FID.

Как воспроизвести или проверить: Выполнить ячейки 41-46 в FULL_MODE и проверить таблицы gan_metrics_df и comparison_df.

Ожидаемый и фактический результат: Ожидаются FID и IS для обеих моделей на одинаковых 10 000 generated images и 10 000 real images. Фактически нет ни значений, ни вывода выполнения.

Рекомендуемое исправление: Рассчитать метрики в FULL_MODE и сохранить таблицы; debug-значения не использовать в отчёте.

AUD-005 - Minor

Источник: ТЗ / сохранение моделей и результатов

Описание: train_gan сохраняет только checkpoint последней эпохи, а generated samples не сохраняются как файлы. Рекомендация ТЗ предлагает сохранять лучшие веса по визуальному качеству и регулярно сохранять samples.

Как воспроизвести или проверить: Проверить содержимое train_gan и функций визуализации.

Ожидаемый и фактический результат: Ожидается сохранение checkpoints и samples, позволяющее вернуться к результату. Фактически snapshots хранятся в памяти и отображаются, а checkpoint записывается только после завершения обучения.

Рекомендуемое исправление: Сохранить финальные сетки и, при необходимости преподавателя, определить и описать критерий выбора лучшего checkpoint.

AUD-006 - Minor

Источник: Качество обучения DCGAN

Описание: Во время шага генератора параметры дискриминатора замораживаются, но сам DCDiscriminator остаётся в train-режиме. Поэтому его BatchNorm running statistics продолжают обновляться на fake-only batch, хотя веса не оптимизируются.

Как воспроизвести или проверить: Проверить train_gan_batch: после set_requires_grad(discriminator, False) вызывается discriminator(generated_images) без временного eval-режима.

Ожидаемый и фактический результат: Ожидается явно выбранное поведение BatchNorm во время шага G. Фактически running statistics DCGAN discriminator изменяются неочевидным образом; Vanilla discriminator BatchNorm не содержит.

Рекомендуемое исправление: Либо временно переводить дискриминатор в eval на шаге G и затем возвращать train, либо оставить текущее поведение, но явно описать его как принятое архитектурное решение.

AUD-007 - Minor

Источник: Воспроизводимость окружения

Описание: Версии torch, torchvision и torch-fidelity не зафиксированы; команда %pip install может со временем установить несовместимое или отличающееся окружение.

Как воспроизвести или проверить: Проверить ячейку 7 с установкой зависимостей.

Ожидаемый и фактический результат: Ожидается повторяемая версия окружения. Фактически устанавливаются последние доступные версии пакетов.

Рекомендуемое исправление: Зафиксировать проверенные версии основных библиотек или вывести их в выполненном notebook.

AUD-008 - Minor

Источник: Надёжность выполнения

Описание: Загрузка CIFAR-10 и pretrained Inception weights зависит от сети, но код не предоставляет отдельного сценария восстановления после ошибки; пояснение есть только в Markdown.

Как воспроизвести или проверить: Запустить notebook в чистой среде без ранее загруженных данных или временно без сети.

Ожидаемый и фактический результат: Ожидается понятное завершение или возможность повторного запуска. Фактически исключение будет показано пользователю, после чего ячейку придётся запускать повторно вручную.

Рекомендуемое исправление: Перед защитой выполнить notebook в чистой Colab-сессии и убедиться, что все загрузки проходят; отдельная программная обработка ошибки необязательна.

Непроверяемые пункты

- Фактическая загрузка 50 000 training и 10 000 test изображений CIFAR-10 - нужен выполненный вывод ячейки 11.
- Корректное отображение 16 исходных изображений и распределения классов - нужны сохранённые outputs ячеек 13-14.
- Успешное полное обучение Vanilla GAN и DCGAN на GPU - нужны журналы 100 эпох и отсутствие tracebacks.
- Фактическая динамика loss_G и loss_D - нужны графики текущего полного запуска.
- Визуальное качество и разнообразие generated samples - нужны сетки начала, середины и конца обучения.
- Наличие или отсутствие mode collapse - нужны фактические generated samples, а не только loss.
- Плавность latent space interpolation - нужны обе сохранённые последовательности из 10 изображений.
- Реальные FID, Inception Score и IS std - нужна таблица torch-fidelity из FULL_MODE.
- Создание checkpoints vanilla_gan.pt и dcgan.pt - нужны сами файлы или вывод путей и успешной загрузки весов.
- Фактическое время обучения и среднее время эпохи - нужна выполненная итоговая таблица на одном устройстве.
- Финальный вывод о лучшей архитектуре - нужен output ячейки 49 после FULL_MODE.
- Полный сценарий Run all в чистом Colab - нужен выполненный notebook без ошибок загрузки и памяти.
- Дополнительные документы и файлы проекта не проверялись по прямому ограничению пользователя; их отсутствие не считается ошибкой.

Лишняя реализация

- Отдельный FAST_MODE с воспроизводимым подмножеством для технической smoke-проверки.
- Автоматические assert-проверки числа изображений, формы batch, диапазона нормализации и форм выходов моделей.
- Подсчёт параметров генераторов и дискриминаторов.
- Сохранение состояний оптимизаторов вместе с весами G и D.
- Измерение полного времени обучения и среднего времени эпохи.
- Автоматическое удаление временных каталогов с изображениями для FID/IS.
- Программное формирование финального вывода из фактических runtime-метрик без заранее заданного победителя.

Опциональные WGAN-GP, Conditional GAN, StyleGAN2-ADA, DDPM и VAE не реализованы, но это не считается недостатком обязательной части.

Результаты проверок

Чтение ТЗ через Get-Content и ConvertFrom-Json: Успешно. Файл ТЗ имеет nbformat 4.0 и содержит 2 ячейки: Markdown с заданием и пустую code-ячейку.

Чтение реализации через Get-Content и ConvertFrom-Json: Успешно. Реализация имеет nbformat 4.5, 50 ячеек: 27 Markdown и 23 code.

Контрольная сумма реализации: SHA256 B8174C1734459BAC6970C82724229904CE708505E09413771187F230B427268E; размер 70 452 байта.

Подсчёт execution_count и outputs: Выполненных code-ячеек 0 из 23; суммарное число outputs равно 0.

JSON-структура и AST-разбор Python: Пройдены: nbformat=4, cells является списком, синтаксических ошибок после исключения Jupyter magic не найдено.

Динамическая проверка архитектур на CPU: Пройдена: VanillaGenerator и DCGenerator создают тензоры (4,3,32,32), оба дискриминатора возвращают (4,1), значения конечны.

Динамическая проверка одного training batch: Пройдена для Vanilla GAN и DCGAN: loss_D и loss_G конечны, backward и шаги Adam выполняются.

Проверка latent interpolation: Пройдена: результат имеет форму (10,128), первая и последняя точки совпадают с исходными latent vectors.

Поиск секретов и внешних доступов через rg: Совпадений для API key, token, password, secret, drive.mount, W&B и Roboflow не найдено.

Полное обучение, загрузка CIFAR-10 и torch-fidelity: Не выполнялись, чтобы не проводить ресурсоёмкий эксперимент вместо автора; их результат должен быть подтверждён outputs notebook.

Непрошедшие проверки ревьюера: Первая попытка AST-команды имела ошибку экранирования, попытка использовать nbformat в bundled Python завершилась отсутствием пакета, а bundled Python не содержал torch. После перехода на JSON/AST стандартной библиотеки и локальный Python с torch проверки прошли. Эти сбои не являются ошибками notebook.

Подготовка к защите

Сценарий демонстрации работы по шагам

1. Открыть notebook в Google Colab, включить GPU и показать версии Python, PyTorch, torchvision и torch-fidelity.
2. Показать, что FAST_MODE=False, EPOCHS=100, используется полный training split и 10 000 изображений для метрик.
3. Загрузить CIFAR-10 и показать размер splits, форму batch и диапазон [-1,1].
4. Показать таблицу распределения классов и сетку из 16 реальных изображений.
5. Показать схему и число параметров VanillaGenerator и VanillaDiscriminator.
6. Показать smoke-check форм и журнал обучения Vanilla GAN.
7. Показать loss-график и snapshots Vanilla GAN на эпохах 0, 50 и 100.
8. Показать схему, инициализацию и число параметров DCGAN.
9. Показать журнал, loss-график и snapshots DCGAN.
10. Показать совместные графики loss и финальные сетки обеих моделей.
11. Продемонстрировать latent interpolation Vanilla GAN и DCGAN по одному latent path.
12. Показать таблицу FID, Inception Score и IS std на 10 000 samples.
13. Показать итоговую таблицу времени, параметров и метрик и объяснить, почему loss не определяет победителя.
14. Завершить конкретным анализом качества, разнообразия, артефактов и возможного mode collapse по полученным изображениям.

Обязательные экраны, команды и тестовые данные

- Экран с GPU, версиями библиотек и строкой FULL/final.
- 50 000 training и 10 000 test изображений; batch shape (64,3,32,32); диапазон нормализации.
- Распределение десяти классов и сетка 4x4 реальных изображений.
- Таблицы параметров Vanilla GAN и DCGAN и результаты smoke-check форм.
- Логи 100 эпох каждой модели.
- Loss_G и loss_D отдельно и на совместном сравнительном графике.
- Snapshots обеих моделей на одинаковом fixed noise.
- Две latent interpolation последовательности по 10 изображений.
- Таблица FID, IS и IS std с Evaluation=full/final и Images=10000.
- Итоговая таблица времени, среднего времени эпохи и числа параметров.
- Файлы vanilla_gan.pt и dcgan.pt либо демонстрация их успешной загрузки.

Доказательства выполнения ключевых требований

- Полностью выполненный notebook с сохранёнными outputs и без tracebacks.
- Логи полного обучения Vanilla GAN и DCGAN.
- Графики loss и сгенерированные сетки начала, середины и конца обучения.
- Latent interpolation для обеих моделей.
- FID и Inception Score на одинаковых 10 000 samples.
- Checkpoint каждой модели.
- Итоговое сравнение времени, параметров и качества.
- Текстовый вывод, основанный на реально полученных числах и изображениях.

10 вероятных вопросов преподавателя и краткие ответы

1. Почему выбран CIFAR-10? Это основной датасет из ТЗ: 60 000 RGB-изображений 32x32 из 10 классов. Его небольшой размер позволяет сравнить две GAN-архитектуры в Colab.

2. Почему изображения нормализуются в [-1,1]? Генераторы завершаются Tanh, поэтому их выход и реальные изображения должны находиться в одном диапазоне.

3. Чем Vanilla GAN отличается от DCGAN? Vanilla GAN разворачивает изображение и использует Linear-слои, а DCGAN сохраняет пространственную структуру через ConvTranspose2d и Conv2d.

4. Почему в дискриминаторе нет Sigmoid? BCEWithLogitsLoss уже объединяет Sigmoid и бинарную кросс-энтропию более устойчивым способом.

5. Как обучаются G и D? Сначала D обучается отличать реальные изображения от detach-копий fake, затем D замораживается, а G обновляется так, чтобы D принимал новые fake за реальные.

6. Зачем нужен fixed noise? Одинаковые latent vectors на разных эпохах позволяют видеть именно прогресс модели, а не разницу случайных входов.

7. Почему нельзя выбирать лучшую модель только по loss? GAN loss отражает баланс двух конкурирующих сетей и не является прямой шкалой чёткости или разнообразия изображений.

8. Что означают FID и Inception Score? Меньший FID означает более близкое распределение признаков real и generated images; больший IS указывает на более уверенные и разнообразные generated samples.

9. Как проверяется latent space? Между двумя latent vectors строится линейный путь из 10 точек; плавные изменения изображений указывают на более непрерывное представление.

10. Какая модель оказалась лучше? В приложенном notebook фактический победитель не подтверждён, потому что outputs отсутствуют и включён FAST_MODE. После FULL_MODE ответ формируется из реальных FID, IS, времени и визуальных результатов.

Что обязательно исправить или подготовить до сдачи

- Установить FAST_MODE=False и выполнить весь notebook в GPU-среде.
- Сохранить notebook со всеми logs, tables, plots и generated images.
- Проверить полный запуск на 100 эпохах и 10 000 изображениях для FID/IS.
- Добавить конкретный визуальный анализ полученных samples и latent interpolation.
- Подготовить checkpoints обеих моделей и подтвердить их успешное создание.

- Убедиться, что итоговый вывод ячейки 49 помечен full/final и основан на таблице текущего запуска.

Решение по приёмке

Обязательно исправить до сдачи

- Выполнить notebook целиком с FAST_MODE=False и сохранить outputs.
- Предъявить обучение обеих моделей на полном training split в течение 100 эпох.
- Предъявить FID и Inception Score на 10 000 real/generated images для каждой модели.
- Сохранить loss-графики, generated samples, latent interpolation и итоговую таблицу сравнения.
- Сформулировать фактический визуальный анализ и вывод о лучшей модели по результатам полного запуска.

Желательно исправить

- Зафиксировать версии torch, torchvision и torch-fidelity.
- Сохранять финальные sample grids как отдельные файлы.
- Явно определить стратегию сохранения лучшего checkpoint, если её требует преподаватель.
- Явно решить поведение BatchNorm дискриминатора на шаге генератора.
- Перед защитой проверить чистый запуск с загрузкой CIFAR-10 и Inception weights.

Можно объяснить как ограничение работы

- Низкое разрешение CIFAR-10 32x32 ограничивает детализацию generated images.
- Обе модели безусловные и не позволяют задавать класс изображения.
- Результат одного seed не описывает полную вариативность adversarial training.
- Время обучения зависит от конкретного GPU текущей сессии.
- WGAN-GP, Conditional GAN, StyleGAN2-ADA, DDPM и VAE не реализованы, поскольку они опциональны.
- Google Drive не используется, так как сохранение туда дано как рекомендация, а не обязательный пункт.

Требуется уточнение у преподавателя

- Нужно ли прикладывать checkpoints отдельными файлами или достаточно выполненного notebook?
- Обязательно ли сохранять промежуточные samples на каждой N-й эпохе или достаточно эпох 0, 50 и 100?
- Требуется ли отдельная ссылка на Colab при сдаче самого Jupyter Notebook?
- Нужно ли реализовывать один опциональный продвинутый GAN для максимальной оценки?
- Нужно ли использовать Google Drive или допустимо скачать результаты из локального runtime после выполнения?